



네트워크_04일차

1. HTTP와 HTTPS의 차이는?
2. GET과 POST의 차이는?

▼ 박해경

▼ HTTP와 HTTPS의 차이는?

HTTP는 서버와 클라이언트 사이에 데이터를 전송하기 위한 프로토콜이자 인터넷에서 **HyperText**를 교환하기 위한 통신 규약입니다.

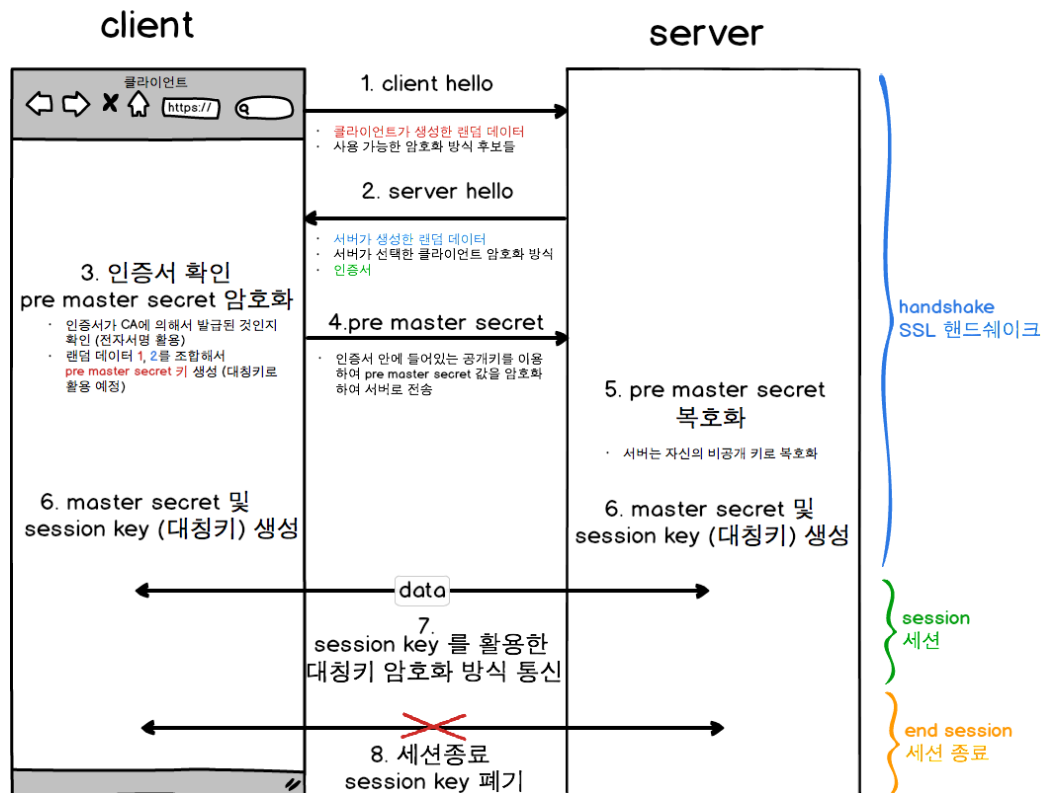
3다만 **HTTP**는 암호화가 되지 않은 평문 데이터를 전송하기 때문에 데이터 유출이 발생할 수 있다는 문제점이 있었고 이를 해결하기 위해 HTTPS가 생겼습니다.

HTTPS는 **HTTP**에 데이터 암호화가 추가된 프로토콜입니다. 네트워크 상에서 중간에 제 3자가 정보를 볼 수 없도록 암호화를 지원하고 있습니다.

HTTPS 연결 과정에서 SSL Handshake가 이루어지며 대칭키 암호화 방식과 비대칭키 암호화 방식이 모두 사용됩니다.

- 클라이언트가 인증서의 공개키로 pre master secret 암호화하고 서버에서 이를 개인키로 복호화는 과정 - 비대칭키 방식
- session key를 활용한 암호화 방식의 통신 - 대칭키 방식

SSL 통신과정



#데이터 전송 프로토콜 #암호화 #SSL Handshake #비대칭키, 대칭키

▼ GET과 POST의 차이는?

GET은 주로 **조회**를 목적으로 사용되며 **요청데이터를 url의 파라미터**로 담아서 사용합니다. 이 방법에는 query string 방식(/user?id=123)과 path value(/users/123) 방법이 있습니다. 이 방식 모두 url에 데이터를 저장하는 방법이므로 **전송하는 길이에 제한이 존재**합니다.

POST는 주로 **데이터를 생성**할 때 사용되며 **요청 정보를 BODY에 담아** 사용합니다. GET과 달리 **길이 제한이 없어 대용량 데이터 전송**이 가능하지만 요청 헤더의 **Content-type**에 요청 데이터의 타입을 정의해야 합니다.

#조회/url 파라미터/길이제한 #생성/BODY/길이제한없음

SSL Handshake

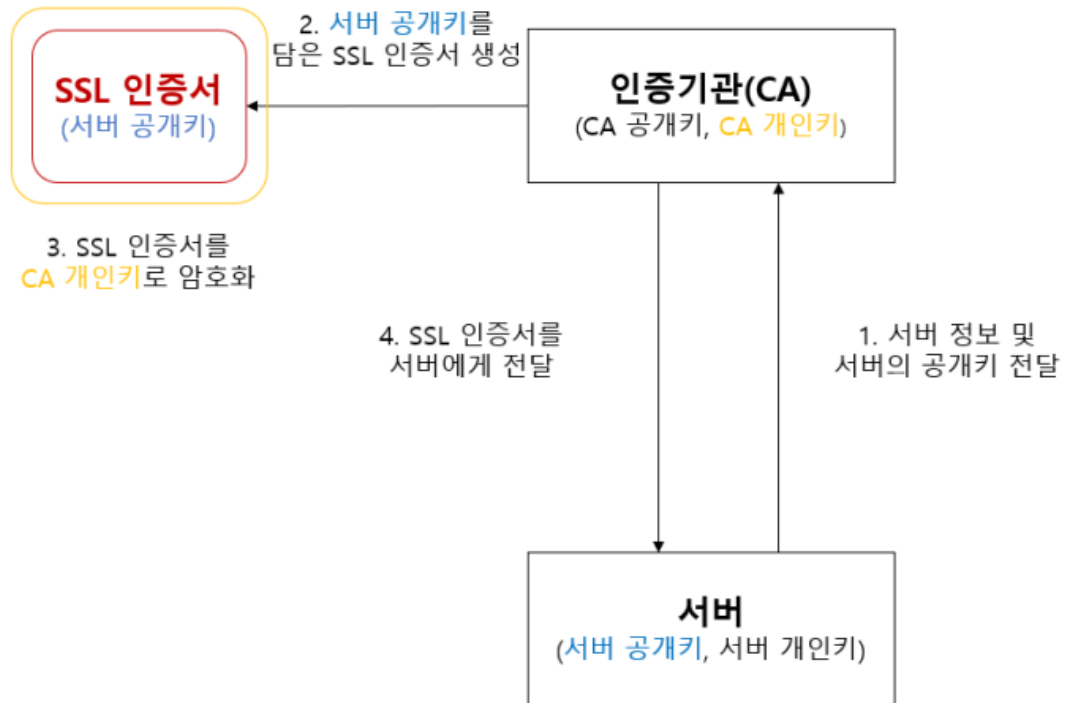
Certificate

Server가 자신의 SSL 인증서를 Client에게 전달합니다. 인증서 내부에는 Server가 발행한 **공개키(+개인키는 Server가 소유)**가 들어있습니다. Client는 Server가 보낸 **CA(Certificate Authority, 인증기관)의 개인키**로 암호화된 이 SSL 인증서를 이미 모두에게 공개된 **CA(Certificate Authority, 인증기관)의 공개키**를 사용하여 복호화합니다. 복호화에 성공하면 이 인증서는 CA(Certificate Authority, 인증기관)가 서명한 것이 맞는 것이니 진짜임이 증명되는 것이죠. 즉, 인증서를 검증하는 것입니다.

또한 Client는 데이터 암호화에 사용할 **대칭키(비밀키)**를 생성한 후 SSL 인증서 내부에 들어 있던 (Server가 발행한) 공개키를 이용해 암호화하여 Server에게 전송할 것입니다. 아래를 보면 SSL 인증서가 무슨 알고리즘(RSA)으로 암호화되었고, 무슨 Hash 알고리즘(SHA256)으로 서명되었는지 확인 가능합니다.

또한 Client는 데이터 암호화에 사용할 **대칭키(비밀키)**를 생성한 후 SSL 인증서 내부에 들어 있던 (Server가 발행한) 공개키를 이용해 암호화하여 Server에게 전송할 것입니다. 아래를 보면 SSL 인증서가 무슨 알고리즘(RSA)으로 암호화되었고, 무슨 Hash 알고리즘(SHA256)으로 서명되었는지 확인 가능합니다.

CA 발급과정



<https://aws-hyoh.tistory.com/39>

- 1). client → server : hello (랜덤 데이터1)
 - 2). server → client: hello (랜덤 데이터2, SSL인증서)
 - 3). client : SSL인증서 확인 with CA 공개키 (CA에 의해 발급된게 맞는지 (CA의 공개키로 인증서를 복호화해서 CA 서명 확인 / 인증서 내의 해시 값이랑 인증서 내용 해시한거랑 같은지 확인) → 맞다면 랜덤 데이터1,2를 이용하여 pre master secret 키 생성 (대칭키로 활용))
 - 4). 인증서 안에 서버 공개키를 이용해 pre master secret 암호화하여 서버로 전송
 - 5). server : 자신의 비공개 키로 복호화
- ⇒ 대칭키 완성